

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “ Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю № 2 Вариант
ДЗ

Выполнил:
Студент группы ИУ5-36Б
Калмыков А.Ю.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Листинг кода

Main.py

```
from operator import itemgetter

class Driver:
    """Водитель"""
    def __init__(self, id, name, salary, garage_id):
        self.id = id
        self.name = name
        self.salary = salary
        self.garage_id = garage_id

class Garage:
    """Автопарк"""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class DriverGarage:
    """Связь водителей и автопарков"""
    def __init__(self, driver_id, garage_id):
        self.driver_id = driver_id
        self.garage_id = garage_id

def build_one_to_many(garages, drivers):
    """Создает соединение один-ко-многим (автопарк-водители)"""
    return [
        (d.name, d.salary, g.name, g.id)
        for g in garages
        for d in drivers
        if d.garage_id == g.id
    ]
```

```
]
```

```
def build_many_to_many(garages, drivers,  
drivers_garages):
```

```
    """Создает соединение многие-ко-многим (автопарк-  
водители)"""
```

```
    many_to_many_temp = [  
        (g.name, g.id, dg.driver_id)
```

```
        for g in garages
```

```
        for dg in drivers_garages
```

```
        if g.id == dg.garage_id
```

```
    ]
```

```
    return [  
        (d.name, d.salary, garage_name, d.id)
```

```
        for garage_name, garage_id, driver_id in
```

```
many_to_many_temp
```

```
        for d in drivers
```

```
        if d.id == driver_id
```

```
    ]
```

```
def get_drivers_with_surname_ending_ow(one_to_many):
```

```
    """Задание Д1: Водители с фамилией,  
оканчивающейся на 'ов'"""
```

```
    result = [  
        (d_name, garage_name)
```

```
        for d_name, _, garage_name, _ in one_to_many
```

```
        if d_name.endswith('ов')
```

```
        if d_name.endswith('ов')
```

```
    ]
```

```
    # Сортировка по двум полям: сначала имя водителя,  
потом название автопарка
```

```
    return sorted(result, key=lambda x: (x[0], x[1]))
```

```
def get_garages_with_avg_salary(garages,  
one_to_many):
```

```

"""Задание Д2: Автопарки со средней зарплатой"""
avg_salaries = []
for garage in garages:
    garage_drivers = [
        item for item in one_to_many if item[2]
== garage.name
    ]
    if garage_drivers:
        avg = sum(item[1] for item in
garage_drivers) / len(garage_drivers)
        avg_salaries.append((garage.name,
round(avg, 2)))
    return sorted(avg_salaries, key=itemgetter(1))

```

```

def get_garages_starting_with_a(garages,
many_to_many):
    """Задание Д3: Автопарки, начинающиеся на 'А'"""
    result = {}
    for garage in garages:
        if garage.name.startswith('A'):
            drivers_in_garage = [
                item for item in many_to_many if
item[2] == garage.name
            ]
            # Сортируем водителей по имени
            driver_names = sorted([item[0] for item
in drivers_in_garage])
            result[garage.name] = driver_names
    return result

```

```

def main():
    """Основная функция для демонстрации работы"""
    # Данные
    garages = [

```

```
Garage(1, "Альтернативный автопарк"),  
Garage(2, "Северный автопарк"),  
Garage(3, "Южный автопарк"),  
Garage(4, "Западный автопарк"),  
Garage(5, "Восточный автопарк"),  
]
```

```
drivers = [  
    Driver(1, "Иванов", 40000, 1),  
    Driver(2, "Петров", 45000, 2),  
    Driver(3, "Сидоров", 38000, 3),  
    Driver(4, "Козлов", 50000, 1),  
    Driver(5, "Новиков", 42000, 4),  
    Driver(6, "Волков", 39000, 2),  
    Driver(7, "Овчинников", 41000, 5),  
    Driver(8, "Медведев", 44000, 3),  
]
```

```
drivers_garages = [  
    DriverGarage(1, 1),  
    DriverGarage(2, 2),  
    DriverGarage(3, 3),  
    DriverGarage(4, 1),  
    DriverGarage(5, 4),  
    DriverGarage(6, 2),  
    DriverGarage(7, 5),  
    DriverGarage(8, 3),  
    DriverGarage(1, 2),  
    DriverGarage(3, 5),  
]
```

```
# Построение отношений
```

```

    one_to_many = build_one_to_many(garages, drivers)

    many_to_many = build_many_to_many(garages,
drivers, drivers_garages)

    # Выполнение заданий

    print("Задание Д1:",
get_drivers_with_surname_ending_ow(one_to_many))

    print("Задание Д2:",
get_garages_with_avg_salary(garages, one_to_many))

    print("Задание Д3:",
get_garages_starting_with_a(garages, many_to_many))

if __name__ == "__main__":
    main()

```

TDD Tests

```

import unittest

from main import (
    build_one_to_many,
    build_many_to_many,
    get_drivers_with_surname_ending_ow,
    get_garages_with_avg_salary,
    get_garages_starting_with_a,
    Garage,
    Driver,
    DriverGarage
)

class TestControlWork(unittest.TestCase):
    """Тесты для рубежного контроля №1"""

    def setUp(self):
        """Подготовка тестовых данных"""

```

```

self.garages = [
    Garage(1, "Альфа-парк"),
    Garage(2, "Бета-парк"),
    Garage(3, "Альтернативный автопарк")
]

self.drivers = [
    Driver(1, "Иванов", 30000, 1),
    Driver(2, "Петров", 40000, 1),
    Driver(3, "Смирнов", 50000, 2),
    Driver(4, "Медведев", 35000, 3),
    Driver(5, "Овчинников", 45000, 3)
]

self.drivers_garages = [
    DriverGarage(1, 1),
    DriverGarage(2, 1),
    DriverGarage(3, 2),
    DriverGarage(4, 3),
    DriverGarage(5, 3),
    DriverGarage(1, 3) # Иванов работает в
двух парках
]

def test_drivers_surname_ending_ow(self):
    """Тест Д1: Фамилии, оканчивающиеся на
'ов'"""

    one_to_many = build_one_to_many(self.garages,
self.drivers)

    result =
get_drivers_with_surname_ending_ow(one_to_many)

    # Все водители, чьи фамилии заканчиваются на
"ов":

    # Иванов, Петров, Смирнов, Овчинников
    expected = [

```

```

        ("Иванов", "Альфа-парк"),
        ("Овчинников", "Альтернативный
автопарк"),
        ("Петров", "Альфа-парк"),
        ("Смирнов", "Бета-парк")
    ]

    # Сортировка по фамилии, затем по названию
    автопарка
    self.assertEqual(result, expected)

def test_garages_avg_salary(self):
    """Тест Д2: Средняя зарплата по автопаркам"""
    one_to_many = build_one_to_many(self.garages,
self.drivers)
    result =
get_garages_with_avg_salary(self.garages,
one_to_many)

    # Расчет:
    # Альфа-парк: (30000 + 40000) / 2 = 35000.0
    # Альтернативный автопарк: (35000 + 45000) /
2 = 40000.0
    # Бета-парк: 50000.0
    expected = [
        ("Альфа-парк", 35000.0),
        ("Альтернативный автопарк", 40000.0),
        ("Бета-парк", 50000.0)
    ]

    self.assertEqual(result, expected)

def test_garages_starting_with_a(self):
    """Тест Д3: Автопарки, начинающиеся на 'А'"""

```



```
        many_to_many = build_many_to_many(
            self.garages,
            self.drivers,
            self.drivers_garages
        )

        result =
get_garages_starting_with_a(self.garages,
many_to_many)

        # Для Альфа-парка: Иванов, Петров
(отсортированы по алфавиту)

        # Для Альтернативного автопарка: Иванов,
Медведев, Овчинников (отсортированы)

        expected = {
            "Альфа-парк": ["Иванов", "Петров"],
            "Альтернативный автопарк": ["Иванов",
"Медведев", "Овчинников"]
        }

        self.assertEqual(result, expected)

if __name__ == "__main__":
    unittest.main()
```

Скриншоты работы программы

```
PS C:\Users\vrema\Documents\GitHub\IU5_3sem_PiKYAP\RK2> & C:/Users/vrema/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vrema/Documents/GitHub/IU5_3sem_PiKYAP/RK2/tests.py
...
-----
Ran 3 tests in 0.001s

OK
PS C:\Users\vrema\Documents\GitHub\IU5_3sem_PiKYAP\RK2> █
```